

0068 — A conflict's shape is read from `kind`, never inferred from the deletion flags

- **Status:** Accepted
- **Date:** 2026-08-23
- **Issue:** #430 (M4.31d), under #84 (M4.31)
- **Supersedes:** nothing. Extends ADR 0063 (stage states) and ADR 0066 (inspecting a conflict) to the *sentence* a user reads.

Context

`NotTextResolvable` already distinguished `Binary`, `Deletion` and `Rename` on the wire. #430 asked for each to get its own surface, because none of the three can be resolved by picking lines, and offering a text resolver for them offers an action that cannot work.

Two facts discovered while building it changed the shape of the answer.

The typed reason never reached the renderer.

`ConflictPanels::open()` accepted a `ConflictedFile` carrying `not_text_resolvable` and returned a struct without it. The reason existed on the wire and died at the display boundary, so no renderer could tell a binary conflict from a text one. The visible symptom was small — a binary pane printed `Binary file (4096 bytes)`, a size rather than an explanation — but the cause was structural.

The deletion flags do not mean what their names suggest.

The server sets `ours_deleted` for `DeletedByUs`, `BothDeleted` and `AddedByThem`; and `theirs_deleted` for `DeletedByThem`, `BothDeleted`

and AddedByUs (`crates/git-vista-server/src/conflicts.rs:164-171`). From the index's point of view "this side has no stage" looks identical whether the side deleted an existing file or never had one. `AddedByThem` is `UA` — *they added it, we haven't touched it*.

A first implementation branched on those two booleans and produced, for `UA`:

We deleted this file; they changed it.

Nobody deleted anything. Nobody changed anything, because there was nothing there to change. **Two facts asserted that the wire never carried.** It reached a commit and was caught by an independent honesty review, not by the test suite — no test exercised the `AddedByUs / AddedByThem` shapes.

Decision

1. `ConflictPanes` carries a `ResolutionSurface`. The classification — a plain-language note, whether a line resolver may open, and a per-control `Result<(), Withheld>` — is computed in `crates/git-vista/src/features/conflicts/core.rs`, which is framework-free and host-tested. The viewer draws what it is handed and decides nothing.

This follows ADR 0066's placement argument exactly: acceptance criteria that are facts about *rendering* must not live in `#[cfg(target_arch = "wasm32")]` code, because `cargo test` never compiles it and the criteria end up pinned by nothing beside a green gate.

2. The deletion sentence branches on `ConflictedFile::kind`, never on the two booleans. `kind` is git's own porcelain classification and cannot conflate "never had it" with "deleted it".

Five kinds get five sentences; `BothAdded` and `BothModified` get a shape-only sentence that names no side.

3. No sentence claims the surviving side changed anything. The wire carries deletion flags, not modification flags. The surviving side is described by what is known — its stage is `Present`, so it "still has it".

4. A control that cannot succeed is replaced by its reason, not disabled. `ConflictedFile::refuses` answers `TakeOurs/TakeTheirs` against an `Absent` stage with `SideAbsent` (`crates/git-vista-protocol/src/conflict.rs:343`). The viewer previously rendered all three controls unconditionally, so pressing "Take theirs" on the side that deleted the file produced a 409. The surface now mirrors `refuses` and says which side and why.

5. An unreadable stage withholds every control, deletion included. `ConflictedFile::all_sides_readable`'s own doc says a caller "must not present a resolution UI for such a file"; nothing enforced it. Deletion is withheld too — it needs no readable stage, so it *looks* safe, but the user would be destroying a file they were never able to inspect.

6. Acceptance criterion 3 — "a rename conflict names both paths" — is NOT implemented, and is recorded here as unbuildable rather than satisfied.

Why rename cannot be built

Not "was not built". Cannot, with what git provides:

- **Nothing constructs `NotTextResolvable::Rename`.** Every occurrence in the tree is the definition, its doc comments, or the field on `ConflictedFile`.

- **git status --porcelain=v2 cannot carry it.** A conflicted path is a `u` record, whose grammar is `u <XY> <sub> <m1> <m2> <m3> <mW> <h1> <h2> <h3> <path>` — **one** path field. `origPath` exists only on `2` (rename/copy) records. Verified against git 2.43's own man page, not only against this repo's parser.
- **The type cannot hold it.** `ConflictedFile` has one `path: String`; `Rename` needs two.
- **Only a heuristic could produce it.** Rename detection is a diff-time similarity guess, not stored state. Reconstructing it would mean re-running that heuristic against per-operation refs (`MERGE_HEAD`, `CHERRY_PICK_HEAD`, `REVERT_HEAD`) whose read paths this repo only partly has.

Presenting a similarity guess as "the other path" would state an inferred fact as an observed one — the precise collapse ADR 0063 exists to prevent, and the same mistake the deletion sentence made above.

If it is wanted later it is its own issue, owning: resolving the correct "theirs" ref per operation, choosing and documenting a similarity threshold, a protocol change so a conflicted entry can carry two paths, and UI language that visibly marks the result as **inferred**.

Alternatives considered

Classify in the viewer. Rejected: `cargo test` never compiles it, so every criterion here would be untested. This is the failure mode #68d and #69c both produced — a fully-tested core with zero consumers beside a green gate.

Disable withheld controls rather than replace them.

Rejected: a greyed button with no explanation is a dead end the user keeps pressing. The commit menu already sets the precedent — it greys impossible operations *with their reason*.

Keep the booleans and special-case the add kinds.

Rejected: it leaves a representation whose field names are actively misleading and requires every future reader to know the exception. Reading `kind` needs no exception.

Derive rename paths with `git diff -M`. Rejected for now — see above. It is a defensible future feature, but not one that can be labelled a fact.

Consequences

- A binary conflict says a line merge is impossible and still offers both whole sides, which the server accepts.
- A delete/modify conflict names the deleting side, and the control for the side holding nothing is withheld with a sentence instead of returning a 409.
- `text_resolution_allowed` is computed and tested but **nothing consumes it yet** — the line-level resolver is #432. It is a tested answer waiting for its caller, not an enforced rule. Said plainly so a later reader does not assume enforcement that does not exist.

- **430's criterion 3 is closed as unbuildable. The milestone does not silently**

claim it.

- One existing test was renamed and narrowed:
`a_delete_modify_conflict_names_which_side_deleted_and_which_changed` was green while guarding the false half of the sentence. A test pinning a claim the data does not support is worse than no test.

Evidence

- 27 host tests in `features::conflicts::core`; 7 adversarial mutations run through `failure-atlas`, **7 caught, no inert tests** — including flipping only `take_deletion` back to `Ok()` inside the unreadable early return.
- 2 browser tests (`ci/browser/tests/nontext-conflicts.spec.mjs`) against a third fixture repository, because `viewer.rs` is wasm-only. Mutation-proved two ways, failing differently: removing the note reddened both tests; un-withholding `take_theirs` left the binary test **green** and reddened only the count assertion — so the two claims are pinned independently.
- A third mutation attempt is recorded as **not applied**: it failed to compile, so the bundle never rebuilt and the resulting red proved nothing. It is not counted.

Signed: max · 2026-08-23T02:2x-04:00